

Points per Micron — `formatscope` Full-Build Execution Playbook (Sep 7-19, 2026)

TL;DR

The full build is feasible in the window **if and only if** you hit four gates in order: INT8 DP32 synthesizes with an area number + cocotb "hello" passes + the model trains (Sep 8); INT4/INT8 units verified with measured accuracies (Sep 10); FP8-E4M3 verified with accuracy (Sep 12); MVP accuracy-vs-area Pareto chart complete (Sep 14). MX formats, OpenSTA timing/power, and the 4×4 tile are strictly stretch work for Sep 15-17 and must never block the core deliverable.

Every tool command, flag, and format definition below is verified against primary sources. The one hard gap: **OpenSTA is NOT bundled in the OSS CAD Suite** — treat critical-path delay and switching-activity power as a fenced stretch requiring a separate OpenSTA build or an OpenROAD Docker image.

The scientific spine is defensible either way: H1 (at equal accumulator width INT8 DP32 is smaller than FP8-E4M3) is backed by Qualcomm's finding that "FP8-E4->32-bit requires 183% more" gates than INT8; H2 (MX shifts the frontier) is testable with the exact OCP MX v1.0 math, fairness rules, exhaustive 256/16-code decoder checks, and 10^4 -vector bit-exact equivalence tests specified here.

Key Findings

Toolchain: The OSS CAD Suite nightly (download asset `oss-cad-suite-linux-x64-YYYYMMDD.tgz`) bundles Yosys, ABC, Icarus Verilog (`iverilog`), Verilator, GTKWave, nextpnr, sby, mcy, eqy, SMT solvers, and cocotb + Python 3.11 — but **not OpenSTA** (confirmed against the suite's own component list). Pin one dated release in `versions.lock`.

cocotb 2.0 changed the API materially and most online tutorials are still 1.x. Verified 2.0 idioms: assign `dut.sig.value = X` (the `<=` shortcut was removed); read `int(dut.sig.value)` or `.to_signed()`/`.to_unsigned()` (`.value.integer` deprecated); `Clock(dut.clk, 10, unit="ns").start()` (no `start_soon` wrapper, keyword is `unit`); runner import `from cocotb_tools.runner import get_runner`; sources via `sources=[...]`; params via `parameters={...}`. Per the cocotb project blog, "cocotb 2.0.0 was released live at ORConf in September [2025], followed by cocotb 2.0.1 this week... cocotb 2.0 is ready for prime time."

Format math is fully pinned by primary specs. OCP OFP8: E4M3 "Max normal $S_{1111110_2} = \pm 2^8 \times 1.75 = \pm 448$ ", bias 7, a single NaN pattern, no infinities; E5M2 " $\pm 2^{15} \times 1.75 = \pm 57,344$ ", bias 15, IEEE-754 special values. OCP MX v1.0: block k=32, E8M0 shared scale (bias 127, `0xFF`=NaN, no Inf), MXINT8 = two's-complement element with implicit 2^{-6} scale, MXFP4 = E2M1 with values {0, 0.5, 1, 1.5, 2, 3, 4, 6}.

Expected accuracy spread justifies CIFAR-10 over MNIST. INT8 PTQ is near-lossless (on ResNet-20/CIFAR-10, "Uniform INT8 improves AI by about 42 percent with virtually no accuracy loss, while uniform INT4... drops accuracy by nearly 2 percent" for a well-tuned scheme); a naive per-tensor INT4 collapses much harder — Table 1 of MetaGrad (arXiv 2303.02347) shows ResNet-20 UINT8-scheme INT4 at **73.45% vs 91.36% FP32 (-17.91 pts)**. FP8-E4M3 "effectively matching the result quality achieved by 16-bit training sessions" (arXiv 2209.05433). MXINT8 "matched baseline FP32 to within the standard deviation on all tasks" (arXiv 2310.10537). MXFP4 shows moderate loss.

H1 has a strong primary anchor. van Baalen et al. (Qualcomm AI Research, arXiv 2303.17951): "FP8-E4->16-bit requires 53% more gates, and FP8-E4->32-bit requires 183% more"; "Compared to INT8, the FP8-E4 format with an FP32 accumulator is 183% less efficient."

Details

0. Repo layout and conventions



```

points-per-micron/
├── README.md
├── pyproject.toml      # or requirements.txt; pinned versions
├── versions.lock      # exact tool + pkg versions (see §1)
├── Makefile           # thin wrappers over python -m and yosys -s
├── .gitignore
├── models/
|   ├── resnet8.py
|   ├── train.py
|   └── checkpoints/    # (gitignored) *.pt
├── quant/
|   ├── formats.py     # decode/encode/quantize for EVERY format — single source of truth
|   ├── fakequant.py   # module replacement / hooks for Conv2d/Linear
|   ├── calibrate.py
|   └── eval.py        # writes results/accuracy.csv
├── rtl/
|   ├── common/        # adder_tree.v, shared params
|   ├── int8/ dp32.v mac.v decode.v
|   ├── int4/ ...
|   ├── fp8e4m3/ ...
|   ├── mxint8/ ...
|   └── mxfp4/ ...
├── tb/                # cocotb tests (Python) + tiny SV wrappers
|   ├── test_decode.py
|   ├── test_dp32.py
|   └── refs.py        # NumPy bit-exact reference models
├── synth/
|   ├── synth.js.template # ONE identical script; {FMT}/{UNIT}/{W} substituted
|   ├── run_synth.py     # renders + runs yosys, parses stat/ltf
|   ├── lib.path         # single file holding the absolute liberty path
|   └── out/             # (gitignored) netlists + logs
├── sta/               # stretch: OpenSTA scripts + reports
|   ├── sta.tcl
|   └── out/            # (gitignored)
├── tool/
|   └── formatterscope/  # CLI package (run/plot/recommend)
├── results/          # COMMITTED: accuracy.csv, area.csv, *.png/svg
└── docs/             # notes, hypotheses, references.bib

```

Conventions: format IDs are the fixed strings `[int4,int8,fp8e4m3,mxint8,mxfp4]`; unit IDs `[dp32|mac|tile]`. Results are CSV (human-diffable) plus a machine JSON mirror. Seeds fixed to 0 everywhere. `.gitignore` covers `**/checkpoints/`, `[synth/out/]`, `[sta/out/]`, `[sim_build/]`, `*.vcd`, `*.fst`, `__pycache__/`, `.venv/`. **Hard rule: no RTL merges to main without its cocotb test green** (enforced by a `make test` pre-merge check).

Makefile targets (thin, so judges reproduce trivially): `make env`, `make train`, `make quant`, `make synth`, `make test`, `make plot`, `make demo`.

README order (matters for the 20% Implementation-Quality and 10% Video rubric lines): one-paragraph pitch → the Pareto chart image → "Reproduce in 5 commands" → results table → H1/H2 with numbers → limitations → references.

1. Day-1 environment setup (verified)

OSS CAD Suite (R + V, every laptop). Go to <https://github.com/YosysHQ/oss-cad-suite-build/releases>, pick one dated nightly, and download the Linux asset named `oss-cad-suite-linux-x64-YYYYMMDD.tgz`. The verified download-URL pattern is <https://github.com/YosysHQ/oss-cad-suite-build/releases/download/<DATE>/oss-cad-suite-linux-x64-<YYYYMMDD>.tgz>.

```
ⓧ
bash
tar xzf oss-cad-suite-linux-x64-YYYYMMDD.tgz
source ./oss-cad-suite/environment # adds all tools to PATH
yosys -V ; iverilog -V ; verilator --version
```

Verified contents (from the suite's component list): **Yosys** (RTL synthesis, "extensive Verilog 2005 support"), **ABC**, **Icarus Verilog** ([iverilog](#)), **Verilator**, **GTKWave**, **nextpnr**, **sby** (formal), **mcy**, **eqy** (equivalence), SMT solvers (Boolector, Yices2, Z3, Bitwuzla), **cocotb**, and **Python 3.11**. **OpenSTA is NOT included** — its component list stops at the tools above and contains no static-timing engine. Pin the exact release date.

WSL2 notes: install everything **inside** the WSL2 distro; keep the repo on the Linux filesystem ([~/points-per-micron](#)), never [/mnt/c/...](#) (small-file I/O there is dramatically slower and cripples the synth/sim loop). YosysHQ: "For best experience on Windows use WSL and linux-x64 version on it."

OpenSTA (stretch, R). Because it is absent from the suite, use one of these verified routes, least-effort first:

OpenROAD — OpenSTA is linked into OpenROAD, so its Docker image / binary (from OpenROAD-flow-scripts) exposes `read_liberty`, `create_clock`, `report_checks`, `report_power`, etc. at the OpenROAD Tcl prompt. Lowest-friction.

Build from source — `git clone https://github.com/parallaxsw/OpenSTA` (the actively maintained upstream; [The-OpenROAD-Project/OpenSTA](#) mirrors it). CMake build. Dependencies: C++ compiler, `tcl`, `swig`, `bison`, `flex`, `eigen`, `zlib` (OpenSTA "uses the zlib library to read compressed Liberty, Verilog, SDF, SPF, and SPEF files"). [GitHub](#)

↗
A documented Docker build avoids dependency hunting.

Do **not** put OpenSTA on the critical path; the Yosys-only area/depth story is the guaranteed deliverable.

Python (all three).



```
bash
python3 -m venv .venv && source .venv/bin/activate
python -m pip install --upgrade pip
pip install "cocotb~=2.0" pytest numpy matplotlib pandas
pip install torch torchvision # CPU or CUDA build as available
```

Verified cocotb facts (docs.cocotb.org, v2.0.0 = pinned, /stable = 2.0.1, APIs identical for everything used here): **pin** `cocotb~=2.0`. Install with `pip install "cocotb~=2.0"` (docs: "pip install "cocotb~=<version>"); requires Python 3.9+ (libpython matching). **pytest is NOT a required dependency of cocotb** (only `find_libpython` is) but you want it as the runner. Verify with `cocotb-config --version`. Recommended path: the **Python Runner** under pytest; Icarus is `get_runner("icarus")`. The runner is officially labeled "experimental," so the classic Makefile flow (`make SIM=icarus`) is your fallback.

⚠ cocotb 2.0 API gotchas (all verified against v2.0.0 docs) — encode these in `tb/`:

Assign: `dut.sig.value = X`. The 1.x `dut.sig <= X` shortcut was removed. Out-of-range assignment raises `ValueError` in 2.0 (was `OverflowError`).

Read/convert: vector reads return a `LogicArray` (not `BinaryValue`). Use `int(dut.sig.value)`, `.to_signed()`, or `.to_unsigned()`. `.value.integer` is **deprecated** (emits a DeprecationWarning). `LogicArray` uses **HDL indexing** — for a `[3:0]` signal, `[0]` is the right-most bit.

Clock: `Clock(dut.clk, 10, unit="ns").start()`. Do **not** wrap in `cocotb.start_soon` (2.0 calls it internally: "versionchanged:: 2.0 `start` now automatically calls `cocotb.start_soon`"). Keyword is `unit=` (singular), not the 1.x `units=`.

Runner: import `from cocotb_tools.runner import get_runner` (not `cocotb.runner`). Use `sources=...` (the deprecated `verilog_sources=` still works but warns). Pass Verilog `parameter` overrides via `parameters={...}`, `define` macros via `defines={...}`, raw sim flags via `build_args=...` (e.g. Icarus `-g2012`). Note the 2.0 change: "defines are no longer implicitly converted to HDL literals" — pass explicit values.

sky130 liberty (R). You need exactly one file: `sky130_fd_sc_hd__tt_025C_1v80.lib` (high-density library, typical-typical corner, 25 °C, 1.80 V). [GitHub](#)

Smallest-download verified route is **volare** (renamed **ciel**), the sky130 PDK version manager, which fetches only requested libraries:



```
bash
pip install volare # or: pip install ciel
export PDK_ROOT="$HOME/.volare"
volare enable --pdk sky130 <open_pdks_commit_hash>
# liberty lands at:
# $PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
```

(volare "supports builds of the sky130 PDK using Open_PDKs, including... sky130_fd_sc_hd"; if a repo has `tool_metadata.yml` you can `volare enable` with no hash.) Verified alternatives if already present:

OpenROAD-flow-scripts `flow/platforms/sky130hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib`, [GitHub](#)

or an `open_pdk` install under `/usr/share/pdk/sky130A/libs.ref/...`. Write the resolved absolute path into a single `synth/lib.path` file that every script reads.

Smoke tests (end of Day 1 — all four must pass).

(a) 4-bit adder area. `rtl/int4/smoke_add.v`:

```
verilog
module smoke_add(input [3:0] a, b, output [4:0] y); assign y = a + b; endmodule

synth/smoke.ys:
```

```
tcl
read_liberty -lib sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog rtl/int4/smoke_add.v
hierarchy -check -top smoke_add
synth -top smoke_add -flatten
abc -liberty sky130_fd_sc_hd__tt_025C_1v80.lib
opt_clean
stat -liberty sky130_fd_sc_hd__tt_025C_1v80.lib
```

Run `yosys -s synth/smoke.ys`. **Look for** a line like `Chip area for module 'Wsmoke_add': <float>` (a positive number). If it prints `0` or the module vanished, the design was optimized away.

(b) cocotb hello on Icarus — minimal DUT + `tb/test_hello.py` via `get_runner("icarus")`. **Look for** `** TEST ... PASS **` and `pytest 1 passed`.

(c) CIFAR-10 one-epoch sanity (Colab T4 or CPU): train ResNet-8 for 1 epoch; **look for** test top-1 climbing above ~40% (well above 10% chance), proving data/loss are wired.

(d) OpenSTA (only if installed): `report_checks` on the adder netlist; **look for** a `Startpoint/Endpoint ... slack (MET/VIOLATED)` path.

2. Accuracy pipeline — exact math and code (owner M)

Model & recipe. Use a ResNet-8-class CNN (stem conv + 3 residual stages) or the canonical ResNet-20 (0.27M params). Known-good recipe reaching ~90–92% on a T4 in well under 20 min: SGD, momentum 0.9, weight decay 5e-4, batch 128, **cosine-annealed** LR from 0.1, ~30–60 epochs (published ResNet-18/CIFAR-10: 0.1 LR + cosine, 200 epochs → ~89% mean; shorter ResNet-8 schedules land ~85–90%). Augmentation: random crop (pad 4) + horizontal flip + per-channel normalization. Keep the FP32 checkpoint as the accuracy ceiling.

Why CIFAR-10, not MNIST. MNIST saturates near 99% at every precision, so all formats look identical and there is no frontier. A small ResNet on CIFAR-10 produces a **measurable spread**: per-tensor INT4 PTQ

visibly hurts while INT8/FP8/MXINT8 stay near-lossless. **Broken-quantizer detector:** accuracy collapsing toward 10% (chance) means a scale/rounding bug, not format cost.

Expected PTQ accuracy table (sanity bounds, with sources).

Format	Expected top-1 vs FP32	Source basis
FP32 baseline	~90–92%	your training run
INT8 (per-tensor)	≈ lossless	ResNet-20/CIFAR-10: "Uniform INT8... virtually no accuracy loss" (arXiv 2512.14090)
INT4 (naive per-tensor PTQ)	large drop (tens of pts possible)	ResNet-20 UINT8-scheme INT4 = 73.45% vs 91.36% FP32, -17.91 (arXiv 2303.02347, Table 1)
FP8-E4M3	≈ lossless	"effectively matching the result quality achieved by 16-bit training sessions" (arXiv 2209.05433)
MXINT8	≈ lossless	"MXINT8 matched baseline FP32 to within the standard deviation on all tasks" (arXiv 2310.10537)
MXFP4	moderate loss	E2M1 coarseness; MX literature

(Note: a well-tuned INT4 scheme recovers most of that loss — MetaGrad's own method reaches 91.2%, -0.16 — but your **naive per-tensor PTQ** is expected to show the large drop, which is the point: it makes the frontier visible.)

Fake-quant implementation. Replace each `nn.Conv2d`/`nn.Linear` with a wrapper that (i) quant-dequants its weight offline once and (ii) quant-dequants its input activation at runtime. Prefer module replacement over hooks for weights. Keep a `quantize_first_last: bool` flag. **Recommendation: quantize everything (first+last included) for honesty**, and if time allows report a second row with first/last kept at INT8. All format math lives in `quant/formats.py` — the *same code* the RTL decoder is checked against in §4.

INT8/INT4 symmetric per-tensor. $s = \max |x| / (2^{(b-1)} - 1)$; $q = \text{clamp}(\text{round_half_to_even}(x/s), -(2^{(b-1)}-1), +(2^{(b-1)}-1))$; dequant $\hat{x} = q \cdot s$. **Choose symmetric ranges** `[-127, +127]` (INT8) and `[-7, +7]` (INT4), dropping the extra negative code (-128/-8) so the quantizer is exactly sign-symmetric — matching the RTL reference and keeping H1's fairness argument clean. Activation calibration: collect per-tensor magnitude over **512 training images** using the **99.99th percentile of |x|** (not raw max) to reject single-sample outliers. Record calibration settings in the CSV.

FP8 E4M3 (OCP OFP8). 1 sign / 4 exp / 3 mant; **bias 7**; normals $v = (-1)^S \cdot 2^{(E-7)} \cdot (1+M/8)$; subnormals (E=0) $v = (-1)^S \cdot 2^{(-6)} \cdot (M/8)$; **no infinities; NaN only** `[S.1111.111]`; **max finite 448** ("Max normal S 1111 110₂ = $\pm 2^8 \times 1.75 = \pm 448$ "). Overflow policy: OCP permits saturate-to-448 *or* overflow-to-NaN — **choose saturate-to-448** (safe for inference, matches AMD/tt convention). E5M2 secondary: bias 15, IEEE-like with Inf/NaN, " $\pm 2^{15} \times 1.75 = \pm 57,344$ ". Encode = round-to-nearest-even on the mantissa with saturation; generate the full 256-entry decode table programmatically and dump it for the RTL check. Per-tensor power-of-two scale so `max|x|` maps just under 448.

MX (OCP v1.0). Block size **k=32** along the reduction (input-channel) dimension; for conv, block along input-channel $\times$ kernel positions flattened. Shared scale $\boxed{X = 2^{\text{shared_exp}}}$, $\boxed{\text{shared_exp} = \text{floor}(\log_2(\max_i |V_i|)) - \text{emax_elem}}$ (emax_elem = max element exponent; E2M1 $\rightarrow$ 2). E8M0 scale: unsigned biased exponent, **bias 127**, range $2^{-127} \dots 2^{127}$, $\boxed{0xFF}$ reserved = NaN, no Inf. Elements:

MXINT8: two's-complement 8-bit element with **implicit 2^{-6} scale** ("the integer data type (i.e., MXINT8) uses a two's complement encoding and has an implicit scale factor of 2^{-6} "); value range ± 1.984375 ; most-negative code ($-128 \rightarrow -2.0$) left unused to keep symmetry.

MXFP4 = E2M1: values {0, 0.5, 1, 1.5, 2, 3, 4, 6} with sign; bias 1; no Inf/NaN; 16 codes.

Block-quantize algorithm: compute $\boxed{\text{shared_exp}}$; divide elements by $\boxed{2^{\text{shared_exp}}}$; round each to the element format (RNE, **saturate/clamp** to element max preserving sign — OCP mandates clamping support during FP4 conversion); reconstruct $\boxed{\hat{x} = 2^{\text{shared_exp}} \cdot \text{element}}$. Edge cases: **all-zero block** $\rightarrow$ set $\boxed{X}$ =E8M0 code 127 ($=2^0$) and all elements 0; **max element rounds up past element max** $\rightarrow$ clamp to element max; **reduction dim not a multiple of 32** $\rightarrow$ zero-pad the final block to 32 (padding contributes 0 to the dot product).

Output: $\boxed{\text{results/accuracy.csv}}$ with columns $\boxed{\text{format, top1, calib_images, calib_percentile, quantize_first_last, seed}}$.

3. RTL specifications — bit-exact and synthesizable (owner R)

General rules (verified): Yosys reads Verilog-2005 natively; use $\boxed{\text{read_verilog -sv}}$ for minor SystemVerilog conveniences; Icarus needs $\boxed{-g2012}$ for SystemVerilog. Flatten 2-D ports to 1-D vectors ($\boxed{a_flat[32*W-1:0]}$) for Yosys/Icarus portability. No latches: every $\boxed{\text{always @*}}$ fully assigns; use $\boxed{\text{localparam}}$; build adder trees with $\boxed{\text{generate}}$; register all outputs so $\boxed{\text{dfflibmap}}$ is exercised identically across the five units.

Common DP32 interface (identical across formats):

```
verilog
module dp32 #(parameter W = 8) (
  input wire      clk,
  input wire      rst_n,
  input wire      en,
  input wire      clear,
  input wire [32*W-1:0] a_flat, // 32 elements, width W each
  input wire [32*W-1:0] b_flat,
  input wire [7:0]   scale_a, // MX only; tie 0 for INT/FP8
  input wire [7:0]   scale_b,
  output reg [31:0]  acc_out
);
```

Registered 32-bit accumulator; combinational multiply + balanced adder tree; **fix one pipeline choice identically across all formats**. Recommendation: purely combinational multiply/add feeding the single

accumulator register (simplest; keeps the fairness argument airtight). `clear` zeroes the accumulator; `en` gates accumulation.

INT8 DP32. 32 signed 8×8 (`signed`) $\rightarrow$ 16-bit products $\rightarrow$ balanced adder tree (5 levels of adds) $\rightarrow$ 21-bit sum $\rightarrow$ sign-extend into 32-bit accumulator. **Accumulator overflow: wraparound (two's complement)** — state it, keep identical across formats. **INT4 DP32:** 32 signed $4 \times 4 \rightarrow$ 8-bit products $\rightarrow$ 13-bit sum.

FP8-E4M3 DP32. Decode each element to sign/exp/mant with hidden bit; handle subnormals ($E=0$); **NaN policy: treat `S.1111.111` as zero and set a sticky flag** (inference data never contains NaN, so this never fires but is documented). 4×4 -bit mantissa product (with hidden bits $\rightarrow$ up to 8 bits). Exponent sum `e_a + e_b - 2 · bias = e_a + e_b - 14`. **Fixed-point accumulator (state exactly):** largest product = `448 × 448 = 200704`; choose a **32-bit two's-complement accumulator with F = 12 fraction bits** so the integer part (up to $\sim 2^{19}$) covers 200704 with headroom and the $\times 32$ lane growth (≤ 5 bits) still fits in 32 bits. Barrel-shifter width = 32 (accumulator width). Saturate on overflow; document the truncated low bits. Provide an **"FP8 $\rightarrow$ FP32-like accumulate" alternative** (float accumulate) but **choose the fixed-point accumulator** because it is bit-exact, deterministic, and directly comparable to INT at equal width — mirroring Qualcomm's "FP8-E4- $\rightarrow$ 32-bit" accumulator configuration.

MXINT8 DP32 / MXFP4 DP32. 32 element multiplies ($\text{INT8} \times \text{INT8}$, or $\text{FP4} \times \text{FP4}$ via a 16-entry LUT that decodes each code to a small fixed-point integer), integer adder tree, then **ONE shared shift** by `(shared_exp_a + shared_exp_b)` relative to a fixed reference exponent, into the 32-bit accumulator. **Clamp the shift to `[0, 31]`** (saturate outside). Fold MXINT8's implicit 2^{-6} per-operand scale (2^{-12} per product) into the fixed-point fraction bits. **Reference exponent** = a fixed constant chosen so the common case needs no negative shift. **E8M0 NaN policy:** if either shared scale = `0xFF`, force the block result to 0 and set the sticky flag.

Single-MAC variants (`mac.v`): one multiplier + accumulator) per format, for the secondary chart. **4 $\times$ 4 tile wrapper** (stretch): 16 `dp32` instances with shared inputs $\rightarrow$ report tile-level area.

Synthesizability checklist: avoid accidental wide `*`; wrap signed operands in `signed()`; register all outputs; `generate` the tree; `hierarchy -check` to catch unconnected outputs early.

4. Verification — exact harness (owner V)

cocotb 2.0 + Icarus under pytest via the Python Runner. Per-file pattern:



python

```

import os
from pathlib import Path
from cocotb_tools.runner import get_runner

def test_dp32_runner():
    sim = os.getenv("SIM", "icarus")
    p = Path(__file__).resolve().parent.parent
    runner = get_runner(sim)
    runner.build(sources=[p/"rtl/common/adder_tree.v", p/"rtl/int8/dp32.v"],
                hdl_toplevel="dp32", parameters={"W": 8},
                build_args=["-g2012"], always=True)
    runner.test(hdl_toplevel="dp32", test_module="test_dp32")

```

Test body skeleton (2.0-correct):

```

❌

python
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge

@cocotb.test()
async def check_vector(dut):
    Clock(dut.clk, 10, unit="ns").start() # no start_soon wrapper in 2.0
    dut.rst_n.value = 0; dut.clear.value = 1; dut.en.value = 0
    await RisingEdge(dut.clk); await RisingEdge(dut.clk)
    dut.rst_n.value = 1; dut.clear.value = 0; dut.en.value = 1
    dut.a_flat.value = packed_a; dut.b_flat.value = packed_b
    await RisingEdge(dut.clk)
    assert int(dut.acc_out.value.to_signed()) == reference

```

Flatten 32 elements little-endian into the `a_flat` int.

Decoder exhaustive tests (`test_decode.py`). A `decode_<fmt>` RTL module outputs the fixed-point/integer interpretation of a code. Loop over **all** codes — FP8: 256, FP4 E2M1: 16, INT8: 256, INT4: 16 — and compare against `quant/formats.py`'s decoder. Exact match required (FP8 NaN codes map to the documented zero+flag policy). Runtime: trivial (<1 s each on Icarus).

DP32 random tests (`test_dp32.py`). NumPy references in `tb/refs.py` mirror the RTL exactly: `np.int64` intermediates, explicit shifts and truncation matching the hardware fixed-point. **10⁴ seeded random vectors per unit**, plus directed corners: all-max, all-min, alternating signs, FP8 subnormal inputs, E8M0 extreme scales, all-zero block, and shift-clamp boundaries. Assert exact equality. Reads use `.to_signed()`/`.to_unsigned()` (never the deprecated `.integer`).

`make test` runs the full pytest suite. For the video, show a green `pytest -q` and the "256/256 FP8 codes match" line.

5. Synthesis — exact Yosys script + metric extraction (owner R)

One identical script per unit (`[synth/synth.ys.template]`, `{FMT}/{UNIT}/{W}` substituted). Every command verified against YosysHQ docs:

```
⊗  
tcl  
read_liberty -lib {LIB}  
read_verilog -sv rtl/common/adder_tree.v rtl/{FMT}/{UNIT}.v  
hierarchy -check -top {UNIT}  
synth -top {UNIT} -flatten  
dfflibmap -liberty {LIB}  
abc -liberty {LIB}  
opt_clean  
tee -o synth/out/{FMT}_{UNIT}_stat.txt stat -liberty {LIB}  
tee -o synth/out/{FMT}_{UNIT}_ltp.txt ltp -noff  
write_verilog -noattr synth/out/{FMT}_{UNIT}_netlist.v
```

Verified command facts:

`stat -liberty <lib>` prints `Chip area for module 'W<top>': <float>` (e.g. the docs' own example `Chip area for module 'Wmain': 65.000000`). `stat -json` outputs machine-readable JSON "to the console; use 'tee' to output to a file." Capture either with `tee -o`.

`ltp -noff` = "print longest topological path"; `-noff` excludes flip-flop paths so you get the combinational depth. This is a **structural gate-depth, not a timed delay** — say so explicitly.

`dfflibmap -liberty` maps flip-flops; `abc -liberty` maps combinational logic; `write_verilog -noattr` emits the clean netlist.

For area comparison, do NOT pass `abc -D <ps>`. The `-D` flag "set[s the] delay target" and would optimize each format toward a delay budget, making areas incomparable. Use `-D` only for a deliberate area-delay tradeoff study. Fairness rule: identical script, liberty, accumulator width, and rounding mode across all five formats.

Nondeterminism: pin the OSS CAD Suite release; ABC can vary, so run each synth 3× and confirm the area is stable (it should be identical for a fixed version + script).

Python parser (`[synth/run_synth.py]`): regex `Chip area for module .*:Ws*([Wd.]+)`; cell count from `Number of cells:`; depth integer from the `ltp` output. Writes `results/area.csv` with columns `format, unit, cell_count, area_um2, ltp_depth`.

Sanity checks. Expected area ordering: INT4 < INT8 ≈ MXINT8 < FP8-E4M3 (H1). Broken-run detection: `stat` showing 0 cells or a tiny area = unit optimized away (outputs unconnected); fix by keeping `acc_out` a registered, used output and ensuring `hierarchy -check` passes.

OpenSTA (stretch only). `[sta/sta.tcl]`:

⊗

```
tcl
read_liberty {LIB}
read_verilog synth/out/{FMT}_{UNIT}_netlist.v
link_design {UNIT}
create_clock -name clk -period 10 [get_ports clk]
set_input_delay -clock clk 0 [all_inputs]
set_output_delay -clock clk 0 [all_outputs]
report_checks -path_delay max -fields {slew cap input nets fanout} -digits 3
report_tns
report_wns
```

All command names verified against OpenSTA docs (`read_liberty`, `read_verilog`, `link_design`, `create_clock`, `set_input_delay`, `set_output_delay`, `report_checks`, `report_tns`, `report_wns`). Derive Fmax from the critical-path arrival (period – max-delay slack). Power: OpenSTA "also supports static power analysis with the `report_power` command"; set switching activity with `set_power_activity` (and it can read a VCD/SAIF). Dump a VCD from Icarus with `$dumpfile`/`$dumpvars` in a small SV testbench wrapper, then `report_power`. **State clearly: these are pre-layout numbers with no wire parasitics** — indicative, not sign-off.

6. The `formatscope` tool and the chart (owner V)

CLI (argparse):

```
formatscope run --formats int4,int8,fp8e4m3,mxint8,mxfp4 --model resnet8_cifar
formatscope plot
formatscope recommend --min-acc 0.87 # smallest-area unit meeting the target
formatscope recommend --area-budget 30000 # best accuracy under the area cap (μm²)
```

Data flow: `run` executes both pipelines (or reads existing `results/accuracy.csv` + `results/area.csv`) and joins on `format`. **Pareto front** = non-dominated set sorted by (area ascending, accuracy descending): keep a point iff no other point has both $\leq$ area and $\geq$ accuracy. **"Knee"** = point maximizing accuracy-per- μm^2 (document this; alternative = largest second-difference). `recommend --min-acc X` $\rightarrow$ min-area format meeting X; `--area-budget Y` $\rightarrow$ max-accuracy format under Y.

Chart spec (finalist quality): x = DP32 area (μm^2 , **linear** primary; `--logx` option), y = top-1 accuracy; one marker per format; Pareto frontier as a step/line; logic depth (`ltp`) shown as marker size **and** annotation; consistent per-format colors across all figures; export PNG **and** SVG at 300 dpi; fonts ≥ 14 pt; no gridline clutter. Second chart: single-MAC area. Third asset: a rendered results-table PNG for the video. matplotlib only.

Live-demo script (`make demo`): runs one INT8 DP32 synthesis (10–60 s on camera) and prints the `recommend --min-acc 0.87` answer.

7. Day-by-day plan (Sep 7–19) with owners and GO/NO-GO gates

Date	M (model/quant)	R (RTL/synth)	V (verif/tool/video)	End-of-day deliverable	Gate
Sun Sep 7	venv + torch; load CIFAR-10; ResNet-8 trains 1 epoch	OSS CAD Suite + liberty; 4-bit adder area smoke	cocotb hello green; repo skeleton + Makefile + CI	Four smoke tests pass	—
Mon Sep 8	Full training → ~90% checkpoint	INT8 DP32 RTL + synth → area printed	INT8 decoder + DP32 harness scaffold	INT8 DP32 area in <code>area.csv</code>	G1: INT8 DP32 synthesizes w/ area; cocotb hello passes; model trains
Tue Sep 9	INT8 + INT4 fake-quant + calibration	INT4 DP32 + single-MACs	INT8/INT4 exhaustive decoder tests green	INT4/INT8 units + decoders verified	—
Wed Sep 10	INT8/INT4 accuracies → <code>accuracy.csv</code>	Fix INT4 tree bugs; re-synth	INT8/INT4 DP32 10 ⁴ -vector tests green	Accuracies + verified INT units	G2: INT4/INT8 verified + accuracies logged
Thu Sep 11	FP8-E4M3 quant + 256-code table	FP8 DP32 RTL (decode + fixed-pt accum)	FP8 exhaustive 256-code decoder test	FP8 unit builds + decoder green	—
Fri Sep 12	FP8 accuracy → CSV	FP8 DP32 synth → area	FP8 DP32 10 ⁴ -vector test green	FP8 verified + accuracy + area	G3: FP8 verified + FP8 accuracy
Sat Sep 13	Buffer / first-last variant	Re-run all synth 3× (nondeterminism check)	<code>formatscope</code> <code>run/plot/recommend</code> MVP	3-format Pareto chart draft	—
Sun Sep 14	Freeze INT/FP8 results	Freeze INT/FP8 netlists + reports	Polished 3-format chart + table PNG	MVP demo-able	G4: MVP chart complete — SAFE FINISH LINE
Mon Sep 15	MXINT8 + MXFP4 quant (block=32)	MXINT8 DP32 (shared-shift) RTL	MX decoder + block-quant unit tests	MX units build + decoders green	—

Date	M (model/quant)	R (RTL/synth)	V (verif/tool/video)	End-of-day deliverable	Gate
Tue Sep 16	MXINT8/MXFP4 accuracies	MXFP4 DP32 + synth; OpenSTA if time	MX DP32 10 ⁴ -vector tests green	MX verified + accuracies + area	If MX slips → ship 4-format
Wed Sep 17	Final accuracy table	4×4 tile (stretch); OpenSTA delay/power	Full 5-format Pareto + MAC + tile charts	Complete results + all figures	Drop tile/OpenSTA freely
Thu Sep 18	Rehearse Q&A	Rehearse live synth demo	Record + edit 3-5 min video	Video v1 done	—
Fri Sep 19	README polish	Netlists/reports committed	Submit with buffer (AM)	Public repo + video submitted	Submit by midday

Daily 15-min sync at start of day: each owner states yesterday-done / today-plan / blockers. **Hard rule: no RTL merged without its cocotb test green.** Sep 21 finalists notified; Sep 22 showcase.

Fallback branches: G1 miss → drop the pipeline register, synth combinational-only. G2 miss → INT-only submission still tells H1's core story via the INT4↔INT8 area spread. G3 miss → present INT4/INT8 + FP8 *area* even if FP8 accuracy calibration is imperfect (area is H1's headline). G4 is the guaranteed-shippable state; everything after is upside.

8. Polish for finalist selection (owner V lead)

README (judge-readable): pitch paragraph → Pareto chart at top → "Reproduce in 5 commands" → results table → H1/H2 with numbers → limitations (130 nm ratios, pre-layout, per-tensor scaling, PTQ only) → references (OCP OFP8, OCP MX v1.0, arXiv 2303.17951, arXiv 2209.05433, arXiv 2310.10537).

Video shot list (3-5 min):

0:00-0:25 Hook: "Which number format gives the most accuracy per micron of silicon?" State H1/H2.

0:25-1:15 Tool live: `formatscope run` → one synthesis on camera → `recommend --min-acc 0.87` prints the winner.

1:15-2:05 The chart: walk the Pareto frontier; point to the INT8-vs-FP8 area gap and where MXFP4 lands.

2:05-2:45 Proof: green `pytest`, "256/256 FP8 codes match," "10⁴/10⁴ vectors exact."

2:45-3:35 Findings: your measured INT8-vs-FP8 DP32 area ratio next to Qualcomm's "FP8-E4-→32-bit requires 183% more" gates; H2 (MX frontier shift).

3:35-4:20 Limitations + next: per-channel scaling, real place-and-route, energy.

Recording tips: 1080p screen capture, terminal font ≥18 pt, narrate ~140 wpm, add captions, keep the live synth clip to a unit that finishes in <60 s.

Showcase Q&A — model answers:

Why a 32-bit fixed-point accumulator for FP8? Bit-exactness, determinism, and direct comparability to INT at equal width — mirrors Qualcomm's "FP8-E4->32-bit" configuration.

FP8 subnormals? Decoded with $E=0 \rightarrow \lfloor 2^{-6} \cdot (M/8) \rfloor$, hidden bit dropped; handled in the decode LUT and checked over all 256 codes.

Why block size 32? It is the OCP MX v1.0 mandated k for all concrete formats (all "use a group size of 32 and the E8M0 format for the shared scales").

Transfer to 5 nm? Absolute μm^2 won't; the **ratios** (INT8 vs FP8) are the transferable claim, though exact ratios shift with library and node.

Per-tensor vs per-channel? Per-tensor is the honest worst case and keeps the hardware comparison clean; per-channel would help INT4 and is future work.

What does `ltp` measure? Longest topological path = structural logic depth in gates, **not** a timed critical path; OpenSTA (stretch) gives actual delay.

How does MXFP4 lose accuracy? E2M1 has only ~3 significand values per binade and one shared exponent per 32 elements $\rightarrow$ coarse representation of in-block outliers.

Why not energy? Pre-layout switching-activity power is indicative only; reported as stretch with the parasitic caveat.

INT8 vs FP8 overhead vs Qualcomm's $\geq 50\%$? Compare your measured DP32 ratio directly to their 50–183% (specifically "FP8-E4->32-bit requires 183% more" gates).

Top 10 pitfalls (detect $\rightarrow$ fix):

cocotb 1.x idioms $\rightarrow$ errors on `<=`/`units=`/`cocotb.runner` $\rightarrow$ use the §1 2.0 API.

`$signed` misuse $\rightarrow$ negatives wrong $\rightarrow$ wrap operands in `$signed()`, test alternating-sign corner.

Latch inference $\rightarrow$ Yosys "inferring latch" warning $\rightarrow$ fully assign every `always @*` branch.

Yosys optimizes logic away $\rightarrow$ `stat` ~0 cells $\rightarrow$ keep outputs registered+used; `hierarchy -check`.

Liberty path wrong $\rightarrow$ `read_liberty` error $\rightarrow$ single absolute path in `synth/lib.path`.

WSL slowness $\rightarrow$ sim/synth crawl $\rightarrow$ keep repo on Linux FS, not `/mnt/c`.

Colab session timeout $\rightarrow$ lost training $\rightarrow$ checkpoint every epoch to Drive; keep epochs modest.

MX block-dim mistake $\rightarrow$ blocking along wrong axis $\rightarrow$ assert block is along the input-channel/reduction dim; unit-test a known vector.

E8M0 bias off-by-one $\rightarrow$ scales $2\times$ off $\rightarrow$ bias is **127**; test `shared_exp=0  $\rightarrow$  code 127`.

ABC nondeterminism $\rightarrow$ area wobbles $\rightarrow$ pin release, run $3\times$, confirm stable.

Recommendations

Days 7–14 are non-negotiable and produce the guaranteed deliverable (INT4/INT8/FP8 verified + Pareto chart). Treat **Sep 14 (G4)** as the point where you already have a finalist-worthy submission even if nothing else lands.

Fence OpenSTA, the 4×4 tile, and the transformer stretch behind G4. They are pure upside; the H1 story needs only area + accuracy.

Put all format math in one file (`quant/formats.py`) and check the RTL against it exhaustively (256 FP8 codes, 16 FP4 codes, 10^4 vectors). This single decision makes the "bit-exact" claim credible and kills the most dangerous class of silent bugs — it directly serves the 20% Results & Evidence and 20% Implementation Quality rubric weights.

Lead the video and README with your measured INT8-vs-FP8 area ratio beside Qualcomm's "183% more" gate figure. That is your strongest Technical-Merit (30%) and AI-and-Chips-Relevance (20%) signal.

Change-the-plan benchmarks: if by **Sep 10** naive INT4 accuracy is not visibly below INT8, your quantizer is broken — fix before proceeding (do not "explain away" a 10%-collapse). If FP8 DP32 area is not clearly above INT8 by **Sep 12**, re-check that the accumulator width is truly identical across formats — that is a fairness bug, not a real result, and would invalidate H1.

Caveats

OpenSTA's absence from OSS CAD Suite is confirmed; the OpenROAD-Docker and parallaxsw/OpenSTA source-build routes are verified to exist but were not executed here — budget real setup time if you attempt them, and keep them off the critical path.

Absolute μm^2 are sky130-specific and **pre-layout** (no wire parasitics); only cross-format **ratios** are the scientific claim.

Published PTQ accuracy figures vary by method/scheme; the table is sanity bounds, not targets. A well-tuned INT4 scheme can recover to near-lossless (MetaGrad reaches 91.2%), so the large INT4 drop you should expect is specifically from **naive per-tensor PTQ** — report your own measured numbers.

The exact ResNet-8 epochs/accuracy depend on your architecture and augmentation; the recipe is a known-good starting point, not a guaranteed top-1.

The cocotb Python Runner is officially labeled "experimental"; the Makefile `SIM=icarus` flow is the fallback if it misbehaves.

cocotb docs list two overflow conventions for FP8-E4M3 (saturate-to-448 vs overflow-to-NaN) — both are OCP-permitted; this playbook fixes **saturate-to-448** so your Python and RTL agree. Document the choice so a judge doesn't read it as a bug.